

PRACTICAL EXAM

Subject Code	SWE202c
Page No	1
Subject Name	Introduction to Software Engineering
Practical Topic	Course Registration System
Time	120 minutes
Tools	MS Word, Draw.io, MS Visio, Astah, Visual Paradigm, Star UML
Note	Open book. Students are NOT allowed to use the Internet.
Student ID	<Fill Student ID here>
Student Name	<Fill Student ID here>

I. CASE STUDY

Green University is a mid-sized institution with approximately 8,000 students and 400 academic staff. Each semester, the university must manage course enrollment for all undergraduate and postgraduate programmers. Currently, students submit paper-based registration forms to their faculty office, and clerks manually enter the data into spreadsheets. This process is error-prone, slow, and creates long queues during the registration window. The university's administration has therefore decided to commission a web-based Course Registration System (CRS) to automate and streamline the entire workflow.

The CRS is expected to serve four main groups of users. Students need to browse the catalogue of available courses, check their eligibility based on completed prerequisites, submit registration requests, and track the status of each request in real time. If a course is fully booked, a student may join an electronic waitlist; should a registered student later drop the course, the system automatically notifies and enrolls the first eligible student on the waitlist. Lecturers need to review the roster of students enrolled in their courses, record midterm and final grades, and release grades to students once the grading period closes. Academic Staff (administrators) are responsible for defining the semester timetable, setting and publishing course quotas, opening and closing the registration window, approving exceptional cases, and generating enrolment and grade-distribution reports. Finally, a System Administrator manages user accounts, configures system parameters, and oversees scheduled data backups.

From a technical standpoint, the university imposes several constraints. The system must remain responsive—handling at least 1,000 simultaneous users during peak registration periods with a page-response time under 3 seconds. Because student records and grade data are sensitive, all communication must be encrypted using TLS 1.2 or higher, and access must be governed by a role-based access control (RBAC) model so that each user sees only the functions and data relevant to their role. The platform must be available 99.5 % of the time during the semester, with any planned maintenance scheduled outside the registration window. The interface must be accessible on both desktop and mobile browsers and must conform to WCAG 2.1 Level AA accessibility standards. The architecture should also support horizontal scaling so that additional server capacity can be added smoothly as student numbers grow in future years.

II. QUESTIONS

Answer ALL questions. Base your answers on the Course Registration System case study above. Where diagrams are required, use standard UML notation and label all elements clearly.

Question 1: Software Development Model (20 points)

1. Identify and justify the most appropriate software development model (SDLC) for the CRS project. In your justification, address each of the following points: (10 points)
 - Whether requirements are fully known and stable at the outset
 - The need for iterative delivery and regular stakeholder feedback
 - Risk factors specific to the CRS (e.g., concurrent-user load, data security)
 - Team size, project duration, and the university's change-control process
2. Select ONE alternative SDLC model and compare it with your chosen model in the context of the CRS. Your comparison must include at least two advantages and two disadvantages for each model. (10 points)

Question 2: UC Modeling (15 points)

1. Identify 4 actors in the Course Registration System. For each actor, state their primary goal and whether they are a primary or secondary actor. (3 points)
2. List at least 6 use cases for the CRS. For each use case, state the initiating actor(s) and a one-sentence description of the goal. (4 points)
3. Draw a complete UML Use Case Diagram for the CRS. The diagram must include: (8 points)
 - All identified actors, placed correctly inside or outside the system boundary
 - All major use cases inside the system boundary rectangle
 - At least ONE <<include>> relationship — provide a brief written rationale
 - At least ONE <<extend>> relationship — provide a brief written rationale
 - Any actor-generalisation relationships that apply

Question 3: UC Specification (15 points)

Write a complete Use Case Specification for the use case 'Register for Course'. Use the template below and fill in EVERY field. Partial or missing sections will receive partial credit only.

UC SPECIFICATION TEMPLATE	
Use Case Name	Register for Course
Use Case ID	UC-03
Actor(s)	Student (primary); System (secondary)
Preconditions	• Student is authenticated and logged in. • Registration window is currently open. • Student has met all prerequisites for the chosen course.

Post conditions	• Student is successfully enrolled in the course, OR • Student is placed on the waitlist if the course is full.
Main Flow (Basic Path)	1. Student selects 'Browse Courses' from the dashboard. 2. System displays the list of available courses with seat availability. 3. Student selects a course and clicks 'Register'. 4. System checks prerequisites → passed. 5. System checks seat availability → seats available. 6. System records the registration and updates seat count. 7. System sends a confirmation notification to the student. 8. Use case ends successfully.
Alternative Flow A	At step 5, if the course is full: - A1. System informs the student that no seats are available. - A2. System offers the option to join the waitlist. - A3. Student confirms → System records the waitlist entry. - A4. Use case ends (waitlisted).
Alternative Flow B	At step 4, if prerequisite check fails: - B1. System displays a message listing the unmet prerequisites. - B2. Student is returned to the course list. - B3. Use case ends without registration.
Exception Flow	At any step, if a network or database error occurs: - E1. System displays an error message and logs the incident. - E2. Student is advised to retry or contact Academic Staff. - E3. Use case ends abnormally.
Business Rules	• A student may not register for courses with overlapping time slots. • Maximum credit load per semester is 24 credit hours. • Waitlist position is determined by timestamp of request.
Non-Functional Requirements	• Response time for registration confirmation must be < 3 seconds. • All data transmitted must be encrypted via TLS 1.2+.

Question 4: Non-Functional Requirements (10 points)

From the case study, identify and classify at least SIX non-functional requirements (NFRs) into appropriate quality categories. For each NFR, write one measurable acceptance criterion. Present your answer as a table with the columns: Category | NFR Description | Acceptance Criterion. (9 points)

Answers

Category	NFR Description	Acceptance criterion

Question 5: Class Diagram (25 points)

1. Identify at least SIX key domain classes for the Course Registration System. For each class, specify at minimum three attributes (with data types) and two methods. Present this as a structured list or mini class-description table before drawing the diagram. (10 points)
2. Draw a complete UML Class Diagram that includes: (15 points)
 - All identified classes with their full attribute and method compartments

- At least ONE inheritance (generalization) relationship — for example, User as a base class for Student, Lecturer, and AcademicStaff
- At least TWO association relationships with correct multiplicity labels at both ends
- At least ONE aggregation OR composition relationship, with the correct diamond notation and a written justification
- At least ONE dependency relationship

Suggested classes (you may add or rename): User, Student, Lecturer, AcademicStaff, SystemAdmin, Course, CourseOffering, Registration, Waitlist, Grade, Semester, Report.

Question 6: Testing Stage and Type (15 points)

1. Map the four standard testing stages to the CRS development process. For each stage, state what is tested, who is responsible, and which testing types are most applicable: (5 points)
 - Unit Testing
 - Integration Testing
 - System Testing
 - User Acceptance Testing (UAT)
2. Write THREE test cases for the 'Register for Course' use case, covering the scenarios listed below. Present each test case in a table with the columns as below : (10 points)
 - Happy-path scenario — a student successfully registers for an available course
 - Boundary / edge-case scenario — a student registers for the last available seat (quota = 1 remaining)
 - Negative / error scenario — a student attempts to register without meeting the course prerequisite

TCID	Test Case Name	Precondition	Test steps	Expected result	Test type
TC1					
TC2					

III. SCORING RUBRIC

Q#	Topic	Max Score	Weight	Grader's Score
1	Software Development Model Selection	2.0	20%	
2	UC Modeling	1.5	15%	
3	UC Specification	1.5	15%	
4	Non-Functional Requirements	1.0	10%	
5	Class Diagram	2.5	25%	
6	Testing Stage and Type	1.5	15%	
TOTAL		10	100%	

IV. SUBMISSION GUIDELINES

4.1. Submission Format

- All work must be submitted as a single .zip file.
- **File naming:** StudentID _SWE202c.zip (e.g., SE123456_SWE202c.zip)
- The .zip must contain: Answer document in MS Word .docx

4.2. Formatting Requirements

- Insert the answers below the question. Each question should begin on a new page where possible.
- Clearly label the question number and sub-question before each answer section.
- UML diagrams must be clear, titled, and use correct standard notation.
- Font: Times New Roman or Arial, size 12–13pt, line spacing 1.5.

4.3. General Evaluation Criteria

- Content accuracy and completeness as required by each question.
- Correct UML syntax, clear and well-annotated diagrams.
- Logical reasoning tied to the specific context of the CRS system.
- Clean, professional presentation.

— *END OF EXAM* —

Label all diagrams clearly and justify your design decisions wherever asked.